

CompArch Final Report

AI Acceleration on FPGA

Kenneth Xiong, Irene Hong

Code Link: <https://github.com/kennethxiong23/ai-fpga>

A major subfield of AI systems research focuses on making model evaluation faster and more efficient. One approach has been building specialized ASICs like Google's TPU, which use highly optimized architectures to accelerate core neural-network operations with extremely high throughput. But ASICs are expensive, power-intensive, and inflexible once manufactured, making them difficult to modify as models evolve. This has led to growing interest in FPGA-based AI accelerators, which offer a middle ground: their reconfigurable hardware allows flexible development and comparably fast computation without the prohibitive cost of designing a custom chip. As a result, FPGAs can deliver competitive speedups for specialized or edge AI workloads while remaining far more adaptable and cost-effective.

This project aims to implement a few fundamental architectures used in FPGA-based acceleration of convolutional neural network (CNN) evaluation, focusing specifically on the convolution layer, the rectified linear unit (ReLU) layer, and the max-pooling layer. Our goal is to design and implement these components and verify that they function correctly in simulation. The rest of the report will focus on the implementation and validation of each layer in simulation. Each section will contain a brief overview of the layer function, the system architecture, and simulation results. Before diving into the specific layers, we will first go over some system requirements/assumptions.

System Overview

We begin by taking black and white images and use a Python script to convert them into a text file that can be read into memory. This Python script recenters each pixel value around 0 and maps it to a range $[-127, 127]$.

Below are some key requirements/assumptions that we made while working on this project:

- The input for our model is a black and white image (one color channel)
- The model we are trying to implement is quantized. This means that certain values have been reduced in complexity/size. Specifically:
 - Float32 weights and biases have been reduced to int8 (signed 8-bit) values
 - Float32 accumulation values(sum/products) are stored as int32 (signed 32-bit) values
- For convolutions, we assume square filters and only one filter per convolution

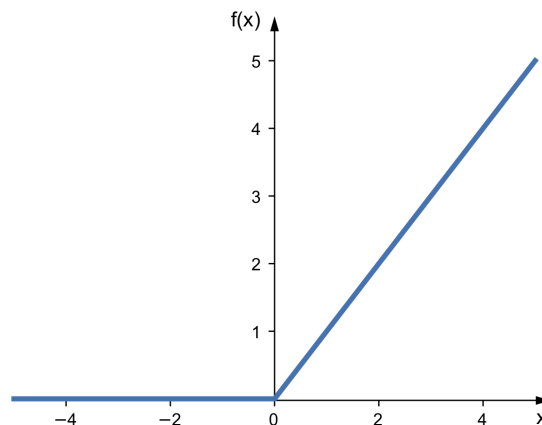
Note: You will notice that when we load in image values, we load them in as 8-bit signed integers, but limit the range of values to be -127 to 127. The signed aspect is crucial for machine learning as it allows us to add nonlinearities (such a ReLU) that are the basis of machine learning. Moreover, we limit the range of values to be -127 to 127, dropping 128, so that the values are centered around 0. Although not relevant for model inference, centering around 0 significantly speeds up model training, so images are confined to that range.

ReLU Activation

For our activation function, we decided to use ReLU, or a rectified linear unit. The activation function is the meat and potatoes of neural networks. These introduce non-linearities into the model, which allow for models to function. Without them, we would not have working machine learning models. ReLU can be described as:

$$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

Graphically, the representation of ReLU is:

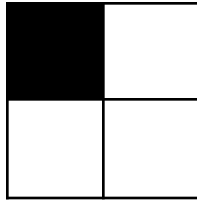


ReLU is also the reason why we need to cast images from $[-127, 127]$. If images are purely in the positive domain, then the ReLU does nothing. However, if they contain some negative values, then ReLU is effective as a non-linearity.

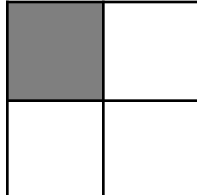
The System Verilog implementation for ReLU is super easy. Given a signed 32-bit integer, we check if it is greater than 0. If it is, we return the value; otherwise, we return 0. We implement a flip-flop block, which controls what value we are looking at. Given an $N \times N$ input, we return a $N \times N$ output, where one ReLU value is computed each clock cycle. This is intentional as it could allow us to set up a pipelined process, where convolutions, activations, and pooling are calculated in the same clock cycle.

Simulation Results

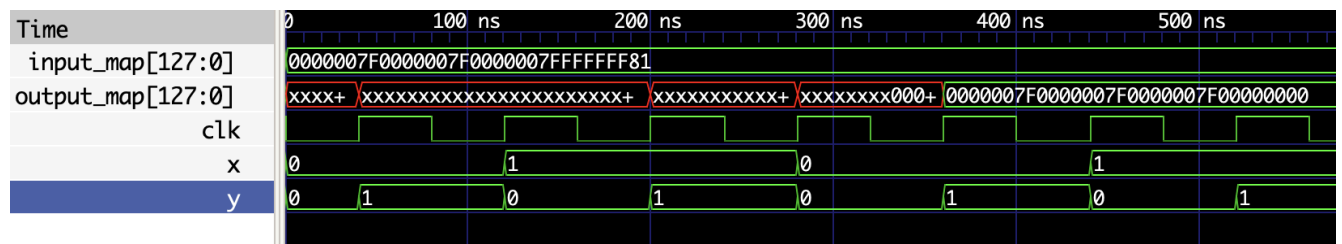
We use the following input feature to evaluate ReLU:



We would expect the output to be:



Here are the results from GTKwave:

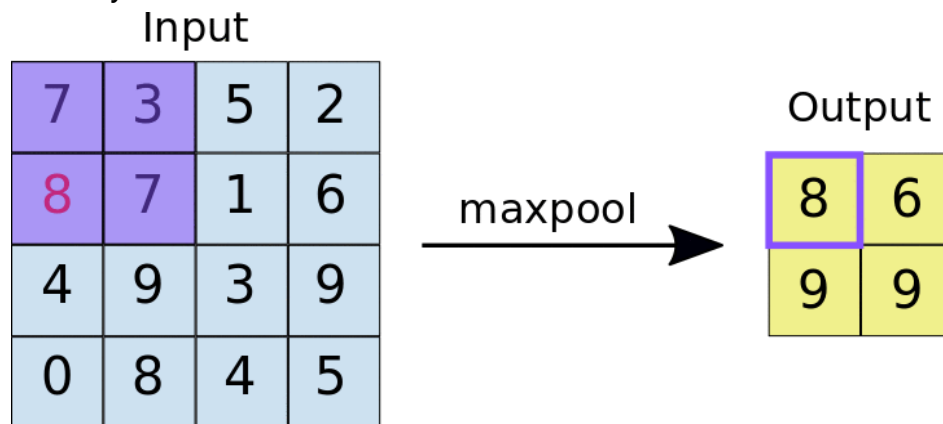


Looking at the output, we see that the -127 value for the black pixel was correctly set to the gray value of 0.

Max Pool

The max pool layer is useful in rapidly reducing the size of the input feature without losing too much relevant information. It can be thought of as taking the maximum of every 2x2 area of the input feature.

Note: Technically, there are other Max Pools besides a 2x2 kernel with a stride (step) of 2; however, this is by far the most common.



Example max pool function over a 2x2 square matrix. [Source](#)

The System Verilog implementation is pretty straightforward. We implement a combinatorial block that calculates the max for a 2x2 region. This is a simple comparison between all four

values. We also implement a flip-flop block, which keeps track of which part of the input we are taking the max of and the corresponding output that it needs to be written (these should be stored in shift registers, but for this project, they are just stored in appropriately sized registers). Each clock cycle, we calculate one maximum value. This is intentional as it could allow us to set up a pipelined process, where convolutions, activations, and pooling are calculated in the same clock cycle.

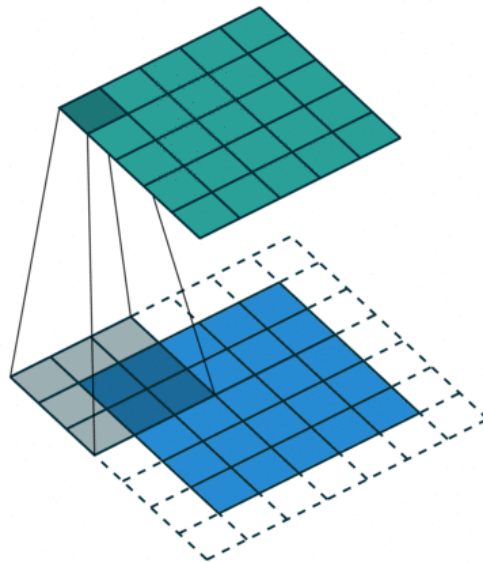
Zero Padding

One thing to consider is the case where we have an odd input. This causes the 2x2 moving frame to not perfectly cover all values. The solution to this is zero padding. When the frame runs over, we fill that area with zeros. Almost surprisingly, this has no impact on model performance and allows a really simple solution.

Convolution

We implement the convolutional layer last, since it builds on some logic from Max Pool. A convolution takes a sliding window (a square window in our case) and adds up values times the weight of that window. There should also be a bias term; this is omitted in our implementation.

Here is a visual representation of the convolution:



Example 3x3 convolution with step 1 performed on an 5x5 input. [Source](#)

The System Verilog is practically identical to that of the max pool. The only changes that we make are that we change the constant stride to a parameter. Also, instead of returning the max value, we have an accumulator that returns a sum at the end of the for loops.

Zero Padding

Similar to max pool, we also account for zero padding. The dashed line sections above show where zeros have been inserted. As a note, usually zero padding is added symmetrically (i.e, if the left and right sides need padding, it is added on the right and left), we only add it as needed.

Simulation Results

We tested a 3x3 convolution with a step size of 3 on the following 5x5 image.

We use the following 3x3 convolution

1	0	1
0	1	0
1	0	1

We would expect the following result:

635	381
381	254

Here are the results from GTKwave:

